APPLICATIONS WEB DISTRIBUÉES

MediConnect

Plateforme intelligente de coordination des services médicaux

Validation 1 · Conception microservices



ÉQUIPE

Rawen Aouina · Sarra Mejbri · Bochra Damak
Rania Ben Haj Hmida · Bechir Belgaied · Yassine Gasmi

PLAN

01 **Sujet et fonctionnalités**
problématique, cas d'utilisation, périmètre

02 **Découpage en microservices**
un service par capacité métier

03 **Architecture, données et sécurité**
schéma global, une base adaptée à chaque service, Keycloak

04 **Technologies**
stack choisie, organisée par couche

05 **Organisation, planning et risques**
Git, Jira

06 **Les microservices en détail**
MS1 à MS7 : rôle, conception, interactions

Contexte, problématique et objectifs

Contexte

Transformation numérique
Le numérique facilite l'accès aux services médicaux.

Services dispersés
Répartis entre établissements et plateformes.

Coordination limitée
Échanges difficiles entre acteurs de santé.

PROBLÉMATIQUE

Comment centraliser les différents services médicaux dans une seule plateforme afin de faciliter leur accès et améliorer la coordination entre les acteurs de santé ?

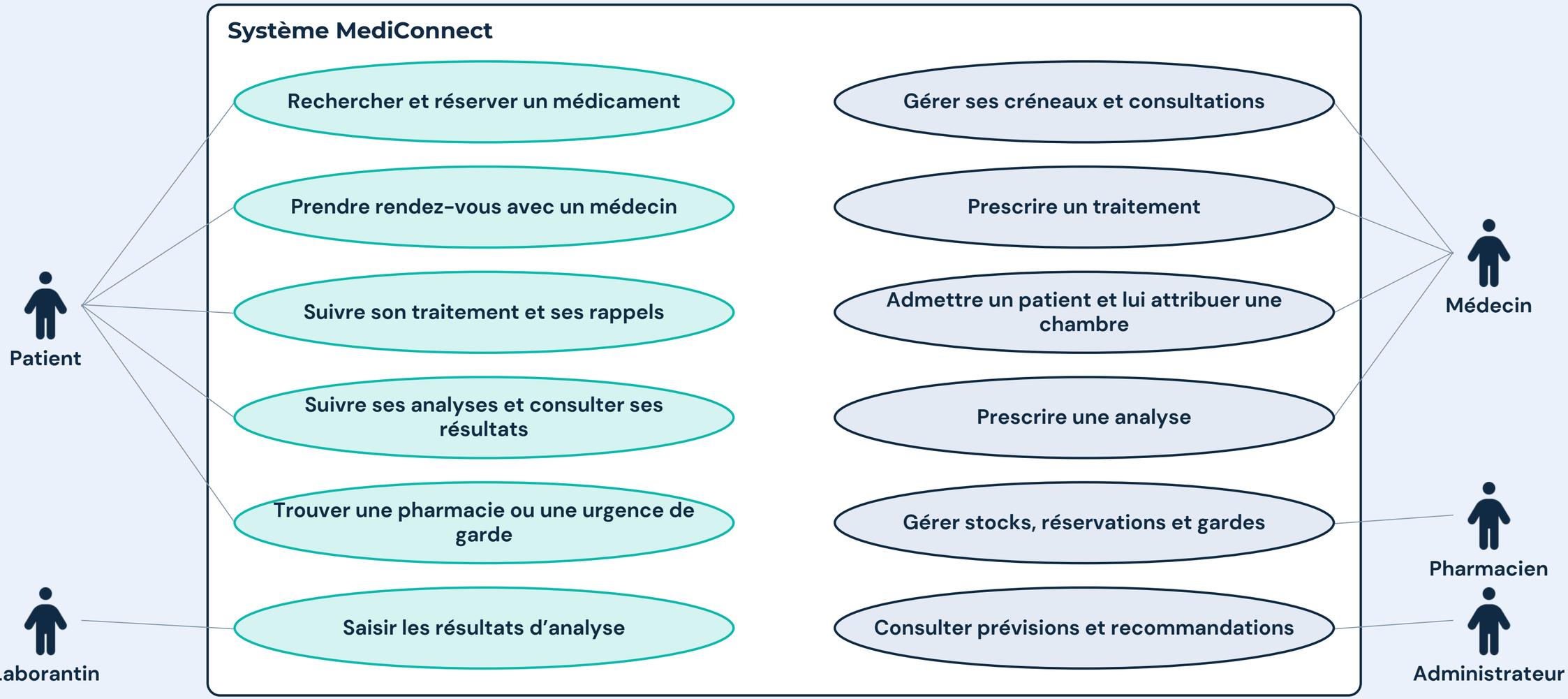
NOTRE RÉPONSE

MediConnect centralise plusieurs services médicaux dans une seule plateforme.

- **Centraliser les services médicaux**
- **Faciliter l'accès aux médicaments**
- **Simplifier les rendez-vous médicaux**

Cas d’utilisation général

Cinq acteurs, douze cas d’utilisation principaux, tous accessibles après authentification Keycloak.



Principales fonctionnalités et périmètre

Inclus dans le projet

1 par service

- Réserver un médicament en pharmacie
- Demander une analyse et suivre ses résultats
- Prendre rendez-vous avec un médecin
- Recevoir les rappels de traitement
- Trouver une pharmacie ou une urgence de garde
- Gérer admissions, chambres et sorties
- Prévoir les besoins et recommander

Hors périmètre

- Diagnostic médical automatisé
- Prescription médicale autonome
- Paiement en ligne
- Téléconsultation (non prévue à ce stade)

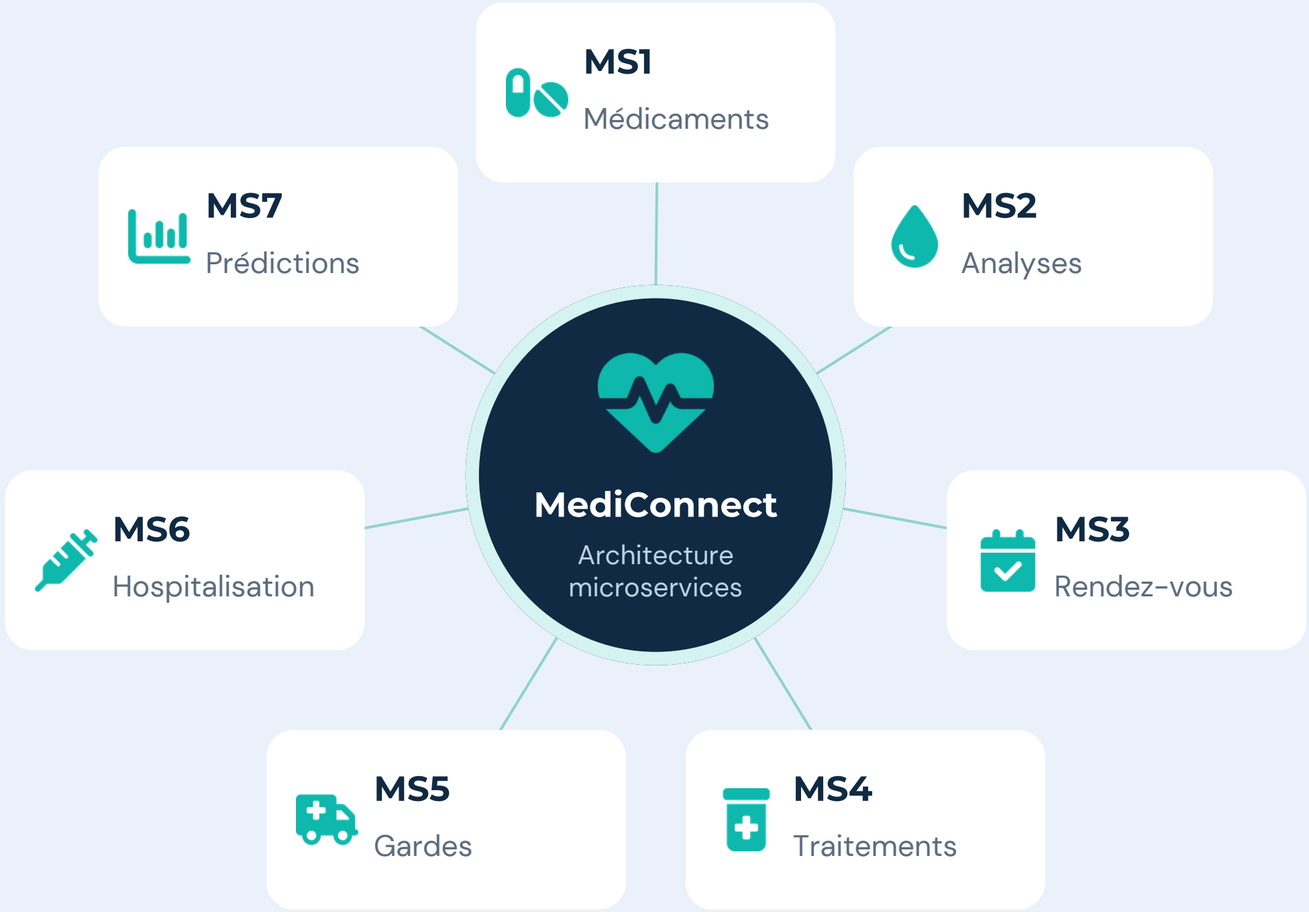
MediConnect facilite l'accès et la coordination des services ; les décisions médicales restent du ressort des professionnels de santé.

Conception et découpage en microservices

Un microservice par capacité métier (contexte borné), sans recouvrement ; MS7 est le service commun.

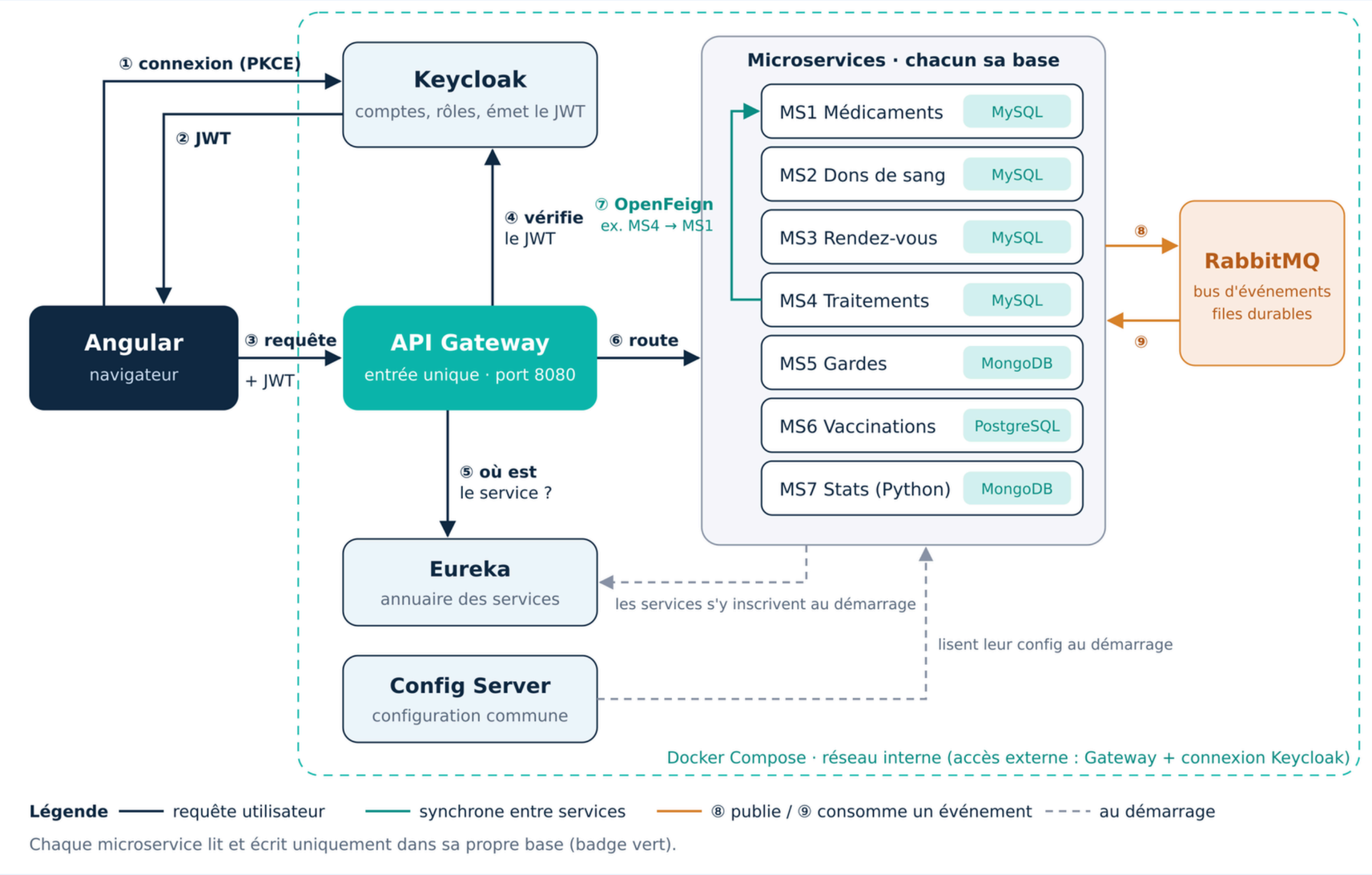
Les 7 microservices

- **MS1** Médicaments & pharmacies
- **MS2** Analyses & laboratoires
- **MS3** Rendez-vous médicaux
- **MS4** Traitements & rappels
- **MS5** Gardes & urgences
- **MS6** Hospitalisation & séjours
- **MS7** Prédiction & recommandation



Architecture cible

Synchrone (OpenFeign) quand la réponse est nécessaire, asynchrone (RabbitMQ) sinon · déployé avec Docker Compose.



Chaque microservice lit et écrit uniquement dans sa propre base (badge vert).

BRIQUES CLÉS

API Gateway
Point d'entrée unique · 8080

Eureka
Découverte des services

Config Server
Configuration centralisée

Keycloak
Authentification et rôles

RabbitMQ
Événements asynchrones

Bases dédiées
MySQL · PostgreSQL · MongoDB

Docker Compose
Déploiement des conteneurs

Gestion des données et sécurité

Chaque service possède ses données ; tout accès passe par la Gateway et Keycloak.

Une base adaptée à chaque service

MySQL / PostgreSQL : transactions ACID · MongoDB : documents et géolocalisation . Aucune base partagée.

MS1 · MySQL
stock et réservations en transaction

MS2 · PostgreSQL
résultats de formats variés (JSONB)

MS3 · MySQL
aucun chevauchement de créneaux

MS4 · MySQL
traitement → prescriptions → prises

MS5 · MongoDB
recherche de la plus proche

MS6 · PostgreSQL
une chambre, un séjour à la fois

MS7 · MongoDB
historique pour les prédictions

Authentification et autorisation

1. **Keycloak** Angular redirige vers Keycloak (Authorization Code + PKCE), qui émet le JWT
2. **Angular** ajoute « Authorization: Bearer <JWT> » à chaque requête
3. **API Gateway** vérifie la signature et l'expiration du jeton (clés publiques Keycloak)
4. **Microservice** revérifie le JWT et contrôle le rôle (@PreAuthorize)

Seule la Gateway est exposée : services et bases de données restent dans le réseau Docker interne.

RÔLES — Patient · Médecin · Pharmacien · Laborantin · Admin

Technologies utilisées

La stack imposée par le module, complétée par un service commun en Python, organisée par couche.

Présentation	**Angular** Interfaces modulaires par domaine	
Accès & sécurité	**Spring Cloud Gateway** Routage, filtres, CORS	**Keycloak + JWT** SSO, rôles et contrôle d'accès
Services métier	**Spring Boot** MS1 à MS6 · REST + Swagger	**Python · FastAPI** MS7 commun · technologie avancée
Infrastructure	**Eureka** Découverte des services	**Config Server** Configuration centralisée **Docker Compose** Conteneurs reproductibles
Échanges & données	**RabbitMQ** Événements asynchrones	**MySQL · PostgreSQL · MongoDB** Une base adaptée par service

Organisation Git et gestion du projet

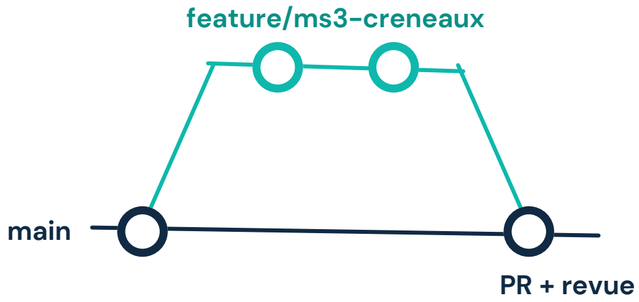
Dépôt public à la structure imposée, un projet par microservice, suivi des tâches dans Jira.

Dépôt Git · structure imposée

- **mediconnect-classe-groupe/**
  - **Backend/**
    - gateway/
    - eureka-server/ · config-server/
    - ms1-medicaments/ … ms6-vaccinations/
    - ms7-statistiques/ (Python)
  - **Frontend/ (Angular)**
  - **Documentation/**
    - Presentation/ · Diagrams/
    - dossier-technique.docx
  - docker-compose.yml
  - README.md

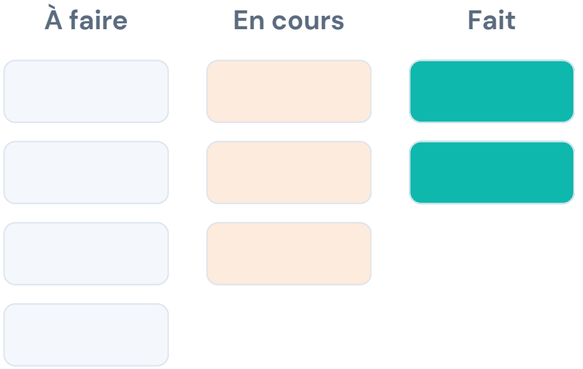
Stratégie de branches

- Branche main protégée
- Une branche par fonctionnalité
- Pull request + revue avant fusion



Gestion · Jira

- Un backlog par microservice
- Un tableau infrastructure
- Revue à chaque séance



Planning prévisionnel et risques

Une étape vérifiable à chaque séance, du squelette à la démonstration finale. (Le développement est organisé en plusieurs étapes)

S4 — Aujourd'hui — S5 — S6 — S7 — S8 — S9 — Fin

Validation 1
Conception validée, projets des 7 services créés

Infrastructure
Eureka, Config Server, routage Gateway

Métier
Endpoints principaux, une base dédiée par service

Sécurité
Keycloak + JWT, appels OpenFeign

Événements
RabbitMQ, service MS7, écrans Angular

Intégration
Docker Compose, tests de bout en bout, README

Soutenance
Démonstration du parcours complet

PRINCIPAUX RISQUES ET PARADES

Contrats qui changent
→ Swagger figé dès S4

Panne d'un service appelé
→ Erreur claire, pas de blocage

Intégration tardive
→ Intégrer à chaque séance

Charge inégale
→ Suivi Jira par membre

MS1 — Médicaments et pharmacies

Spring Boot

Base MySQL dédiée

Rôle



Gérer le catalogue des médicaments, leur disponibilité en pharmacie et les réservations.

Fonctions clés

- Catalogue des médicaments
- Disponibilité par pharmacie
- Réservation de médicaments
- Gestion des stocks

Données clés

- Médicament
- Pharmacie
- Stock
- Réservation

Stockées dans la base MySQL de MS1

Objectif

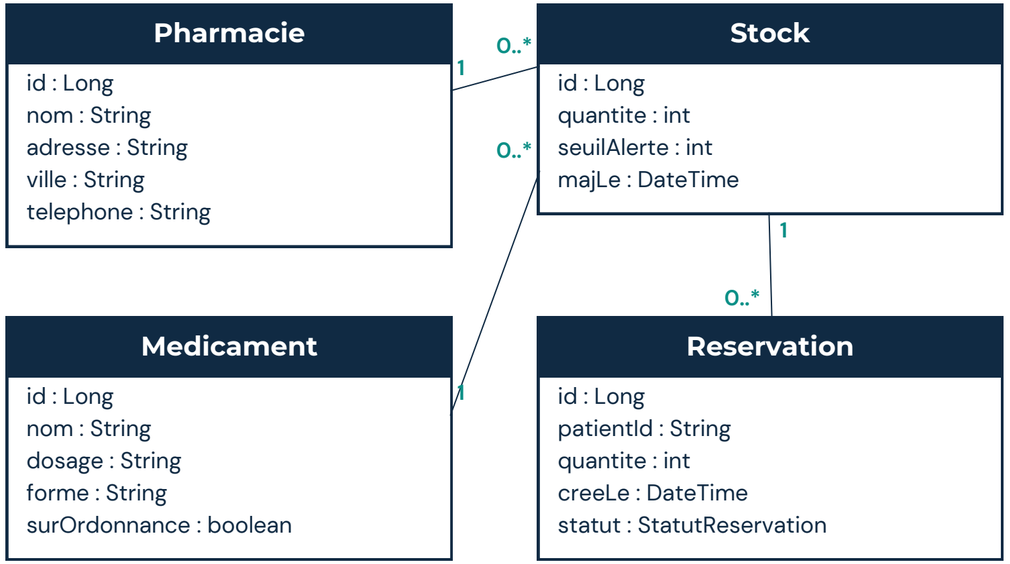
Faciliter l'accès aux médicaments.

MS1 — Conception : classes et API

Spring Boot

Base MySQL dédiée

Diagramme de classes



«enum» StatutReservation

EN_ATTENTE · CONFIRMEE · RETIREE · ANNULEE

Endpoints et codes

GET **/api/medications?nom=**
200 · 400

GET **/api/medications/{id}/availability**
200 · 404

POST **/api/reservations**
201 + Location · 400 · 409

PATCH **/api/reservations/{id}**
200 · 404 · 409

ÉVÉNEMENTS

Publie → MS7 : ReservationCreee

Appelé par MS4, MS5 · appelle MS7

Choix BD

MySQL : stock et réservation modifiés dans une même transaction (pas de sur-réservation).

MS1 — Interactions

Spring Boot

Base MySQL dédiée

Appelle



MS7 · Prédiction

Synchrone (OpenFeign)
prévision de demande
et risque de rupture

Appelé par · événements

MS4 l'appelle (synchrone)
nom et dosage d'un médicament

MS5 l'appelle (synchrone)
nom, adresse, téléphone d'une pharmacie

Vers MS7 : ReservationCreee
statistiques des réservations

En cas de panne

MS7 indisponible
fiche affichée sans prévision

Réservation enregistrée
même si MS7 est arrêté

Stock déjà réservé
refus clair (409)

Aucun blocage pour le patient

Pourquoi

Synchrone vers MS7 pour la prévision et pour MS4/MS5 qui ont besoin de la donnée, asynchrone pour les événements.

MS2 — Analyses médicales et laboratoires

Spring Boot

PostgreSQL dédiée

Rôle



Gérer les demandes d'analyses, leur suivi en laboratoire et les résultats.

Fonctions clés

- Rechercher les laboratoires
- Demander une analyse
- Suivre l'état des analyses
- Consulter les résultats

Données clés

- Laboratoire
- Demande d'analyse
- Analyse
- Résultat

Stockées dans la base PostgreSQL de MS2

Objectif

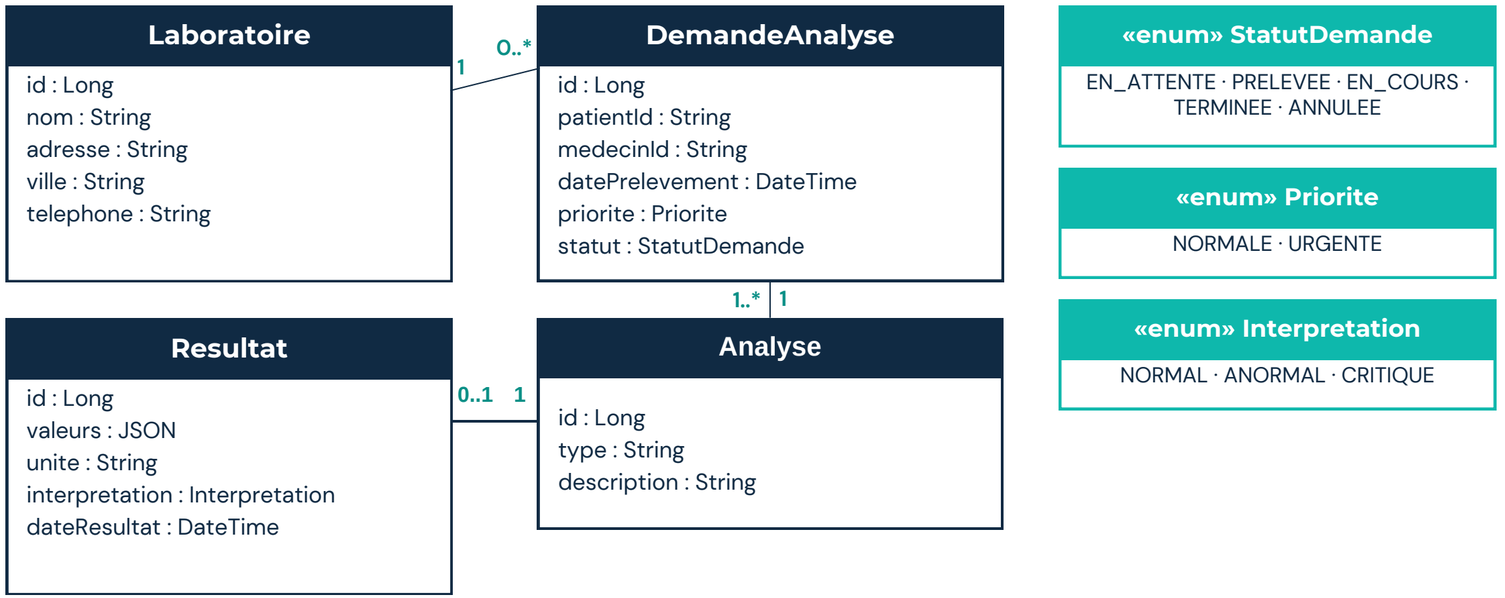
Accélérer l'accès aux analyses et à leurs résultats.

MS2 — Conception : classes et API

Spring Boot

PostgreSQL dédiée

Diagramme de classes



Endpoints et codes

GET	/api/labs?ville=	200 · 400
POST	/api/lab-requests	201 + Location · 400
GET	/api/lab-requests/{id}	200 · 404
POST	/api/analyses/{id}/result	201 · 404 · 409

ÉVÉNEMENTS

Publie → MS4, MS7 : ResultatDisponible

Appelle MS6 en synchrone

Choix BD

PostgreSQL : relations labo → demande → analyses → résultat, et JSONB pour des résultats de formats différents.

MS2 — Interactions

Spring Boot | PostgreSQL dédiée

Appelle

MS6 · Hospitalisation

Synchrone (OpenFeign)
le patient est-il hospitalisé ?

Appelé par · événements

Angular via la Gateway
demandes, suivi, résultats

Vers MS4 : ResultatDisponible
le patient est prévenu

Vers MS7 : ResultatDisponible
statistiques des analyses

En cas de panne

MS6 indisponible
demande créée en priorité normale

MS4 / MS7 indisponibles
événements gardés en file d'attente

Résultat enregistré
normalement dans MS2

Aucun blocage pour le patient

Pourquoi — **Synchrone pour connaître la priorité tout de suite, asynchrone pour prévenir le patient et compter.**

MS3 — Rendez-vous médicaux

Spring Boot

MySQL dédiée

Rôle



Gérer les médecins, les créneaux et les rendez-vous des patients.

Fonctions clés

- Gestion des médecins
- Disponibilités
- Prise de rendez-vous
- Historique des consultations

Données clés

- Médecin
- Patient
- Créneau
- Rendez-vous

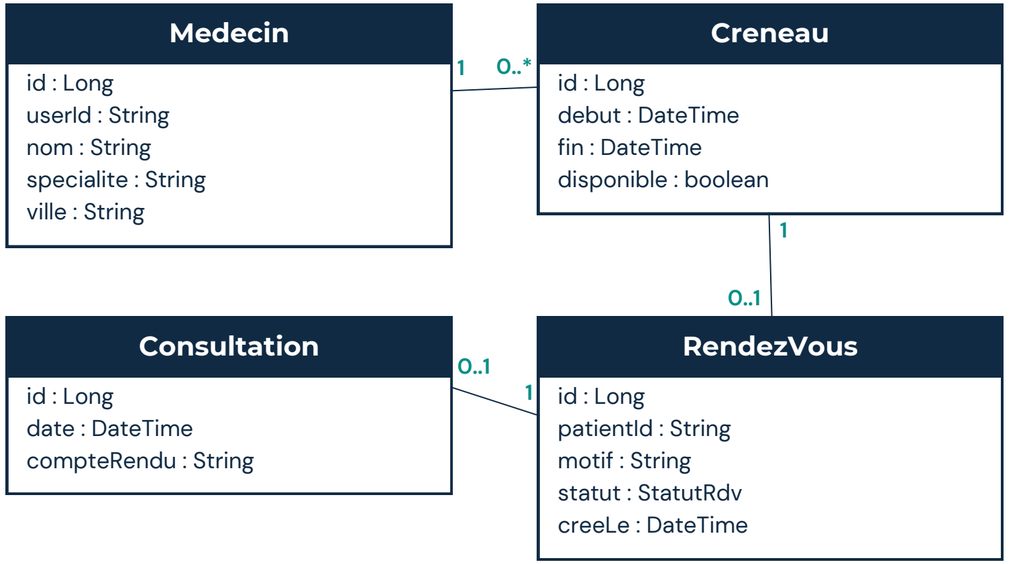
Stockées dans la base MySQL de MS3

Objectif — Simplifier l'organisation des rendez-vous médicaux.

MS3 — Conception : classes et API

Spring Boot | MySQL dédiée

Diagramme de classes



Endpoints et codes

GET	/api/doctors?specialite=&ville=	200 · 400
GET	/api/doctors/{id}/slots	200 · 404
POST	/api/appointments	201 + Location · 400 · 409
PATCH	/api/appointments/{id}/cancel	200 · 404 · 409

ÉVÉNEMENTS

Publie → MS4, MS7 : RendezVousReserve / Annule

Appelle MS4 en synchrone

Choix BD

MySQL : contrainte d'unicité et vérification des créneaux horaires pour empêcher deux rendez-vous d'un même médecin de se chevaucher.

MS3 — Interactions

Spring Boot

MySQL dédiée

Appelle



MS4 · Traitements

Synchrone (OpenFeign)
traitements en cours du patient

Appelé par · événements

MS6 l'appelle (synchrone)
rendez-vous de suivi après une sortie

Vers MS4 : RendezVousReserve
rappel la veille du rendez-vous

Vers MS7 : RendezVousReserve
statistiques des rendez-vous

Vers MS4, MS7 : RendezVousAnnule
rappel supprimé, statistiques à jour

En cas de panne

MS4 indisponible
consultation affichée sans traitements

Message clair
« traitements indisponibles »

Événements
gardés en file d'attente

La réservation n'est jamais bloquée

Pourquoi

Synchrone pour montrer les traitements au médecin, asynchrone pour les rappels et les statistiques.

MS4 — Traitements et rappels

Spring Boot

Base MySQL dédiée

Rôle



Suivre les traitements prescrits et rappeler les prises aux patients.

Fonctions clés

- Création des traitements
- Planification des prises
- Rappels
- Historique du suivi

Données clés

- Traitement
- Prescription
- Rappel
- Historique

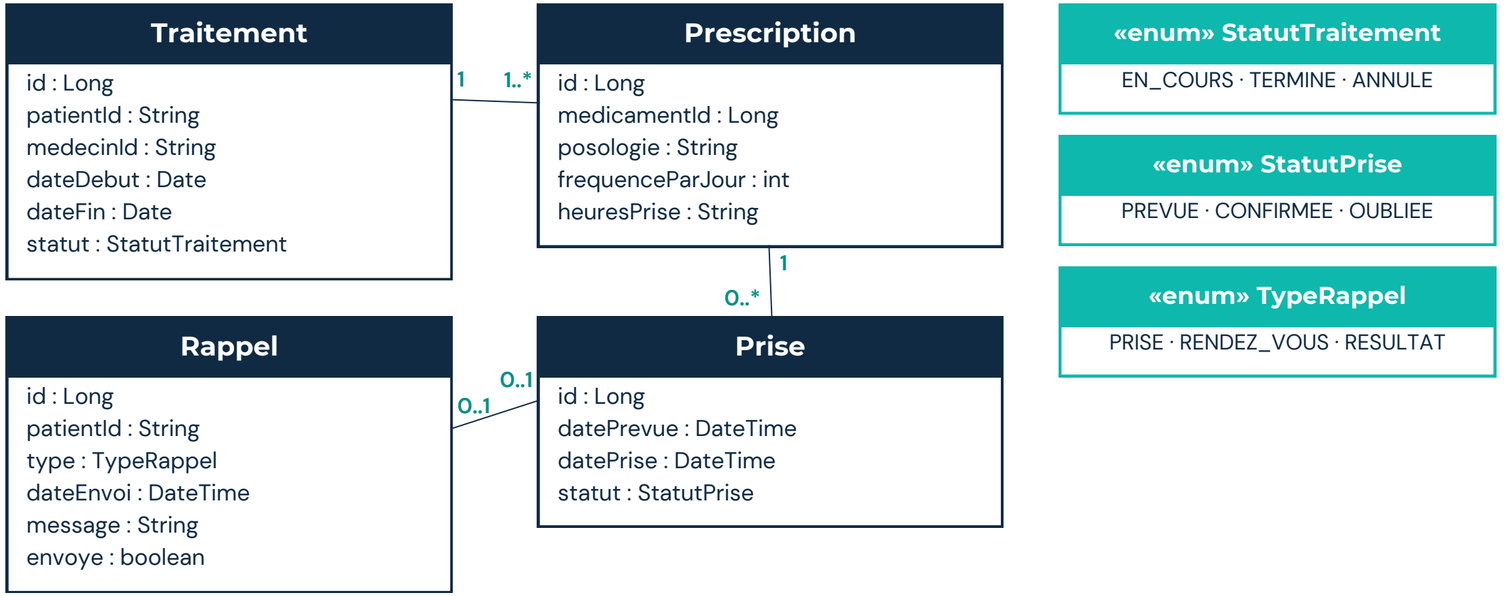
Stockées dans la base MySQL de MS4

Objectif — **Améliorer le suivi des traitements.**

MS4 — Conception : classes et API

Spring Boot | Base MySQL dédiée

Diagramme de classes



«enum» StatutTraitement
EN_COURS · TERMINE · ANNULE

«enum» StatutPrise
PREVUE · CONFIRMEE · OUBLIEE

«enum» TypeRappel
PRISE · RENDEZ_VOUS · RESULTAT

Endpoints et codes

POST **/api/treatments**
201 + Location · 400

GET **/api/treatments?patientId=**
200 · 400

GET **/api/treatments/{id}/intakes**
200 · 404

PATCH **/api/intakes/{id}**
200 · 404 · 409

ÉVÉNEMENTS

Publie → MS7 : PriseConfirmee, PriseOubliee

Consomme : RendezVousReserve, ResultatDisponible

Choix BD

MySQL : données relationnelles car les données formentune hiérarchie stricte (traitement → prescriptions → prises).

MS4 — Interactions

Spring Boot

Base MySQL dédiée

Appelle



MS1 · Médicaments

Synchrone (OpenFeign)
nom et dosage du médicament

Appelé par · événements

MS3 l'appelle (synchrone)
traitements en cours du patient

MS3 → RendezVousReserve
rappel la veille du rendez-vous

MS2 → ResultatDisponible
le patient est prévenu

Vers MS7 : PriseConfirmee, PriseOubliee

En cas de panne

MS1 indisponible
traitement créé avec l'identifiant seul

Nom affiché
« indisponible »

Événements
gardés en file d'attente

Aucun blocage pour le patient

Pourquoi **Synchrone si la réponse est requise tout de suite, asynchrone pour les rappels.**

MS5 — Gardes et urgences

Spring Boot

MongoDB dédiée

Rôle



Indiquer les pharmacies et services de garde en cas d'urgence.

Fonctions clés

- Pharmacies de garde
- Services d'urgence
- Recherche par zone
- Informations pratiques

Données clés

- Pharmacie (réf. MS1)
- Garde
- Urgence
- Localisation

Stockées dans la base MongoDB de MS5

Objectif

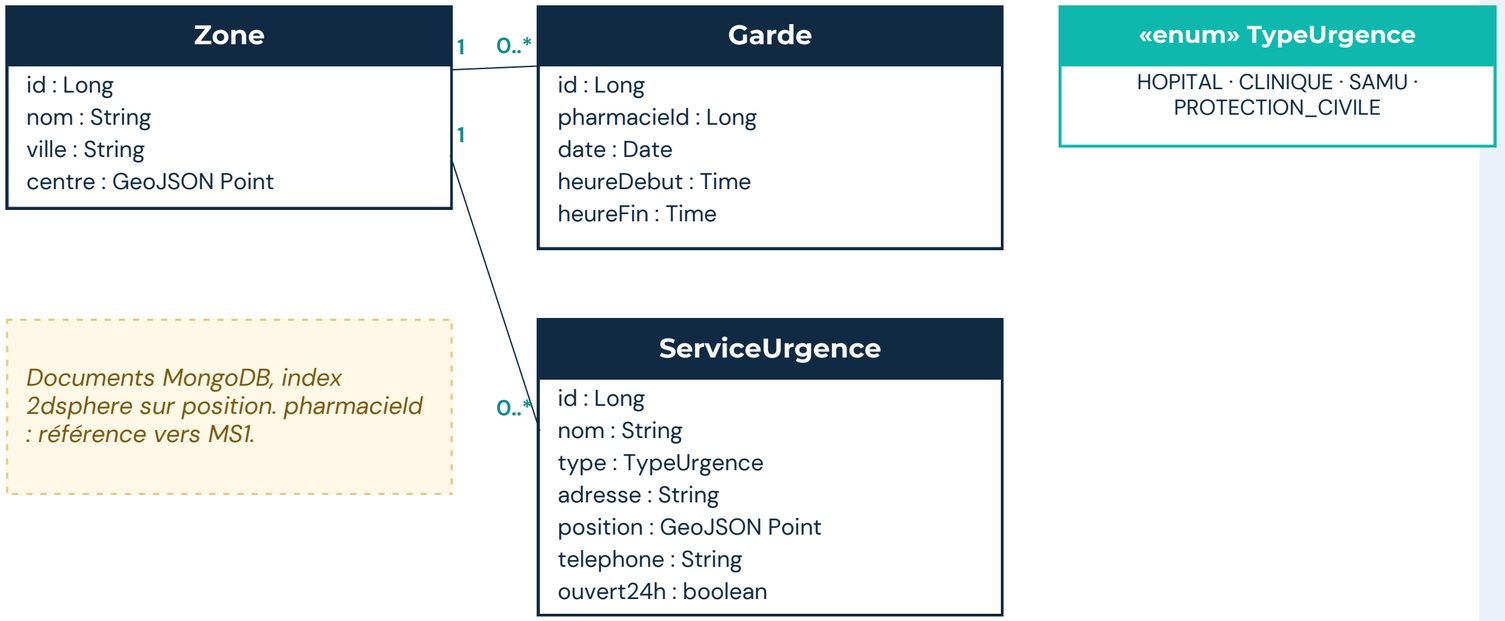
Orienter rapidement le patient en situation d'urgence.

MS5 — Conception : classes et API

Spring Boot

MongoDB dédiée

Diagramme de classes



Endpoints et codes

GET	/api/on-call/pharmacies?zone=&date=	200 · 400
GET	/api/on-call/emergency-services	200 · 400
POST	/api/on-call/shifts	201 + Location · 400 · 409
DELETE	/api/on-call/shifts/{id}	204 · 404

ÉVÉNEMENTS

Aucun événement publié

Appelle MS1 et MS6 en synchrone

Choix BD

MongoDB : index géospatial 2dsphere pour trouver la pharmacie ou l'urgence la plus proche ($near).

MS5 — Interactions

Spring Boot

MongoDB dédiée

communique



MS1 · MS6

Synchrone (OpenFeign)
MS1 : fiche pharmacie
MS6 : chambres libres

Appelé par · événements

Angular via la Gateway
recherche par zone et par date

Aucun événement publié
service de consultation

En cas de panne

MS1 indisponible
garde affichée (zone, horaires)

Nom de la pharmacie
« indisponible »

Urgences toujours affichées
données propres à MS5

Une urgence est toujours trouvée

Pourquoi

Synchrone : pharmacies (MS1) et chambres (MS6) appartiennent à d'autres services, MS5 ne les recopie pas.

MS6 — Hospitalisation et séjours

Spring Boot

PostgreSQL dédiée

Rôle



Gérer les admissions, les chambres et les sorties des patients hospitalisés.

Fonctions clés

- Admettre un patient
- Attribuer une chambre libre
- Suivre le séjour
- Enregistrer la sortie

Données clés

- Admission
- Séjour
- Chambre
- Sortie

Stockées dans la base PostgreSQL de MS6

Objectif

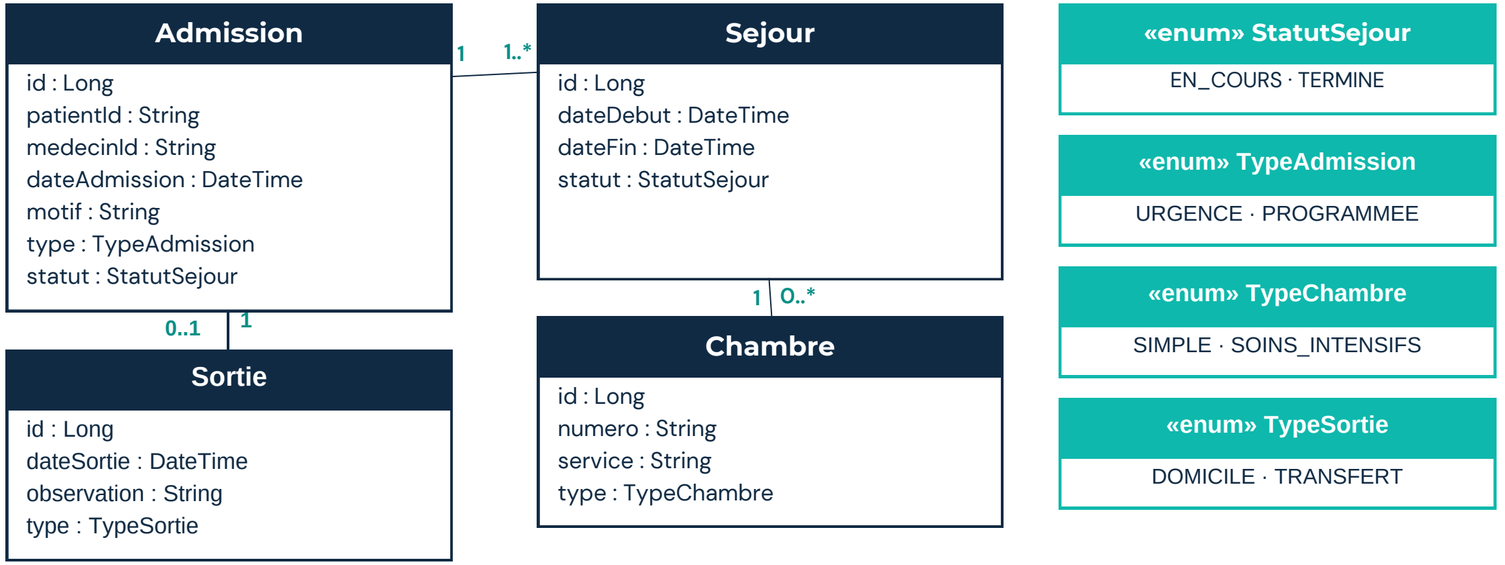
Savoir en temps réel quelles chambres sont libres et qui est hospitalisé.

MS6 — Conception : classes et API

Spring Boot

PostgreSQL dédiée

Diagramme de classes



Endpoints et codes

POST `/api/admissions`
201 + Location · 400

GET `/api/rooms/available?service=`
200

POST `/api/admissions/{id}/stays`
201 · 404 · 409 (chambre occupée)

POST `/api/admissions/{id}/discharge`
201 · 404 · 409 (déjà sorti)

ÉVÉNEMENTS

Publie → MS7 : AdmissionCreee, SortieEnregistree

Appelle MS3 · appelé par MS2, MS5

Choix BD

PostgreSQL : contrainte d'exclusion, deux séjours ne peuvent pas se chevaucher dans la même chambre.

MS6 — Interactions

Spring Boot

PostgreSQL dédiée

Appelle



MS3 · Rendez-vous

Synchrone (OpenFeign)
rendez-vous de suivi
à la sortie

Appelé par · événements

MS2, MS5 l'appellent (synchrone)
patient hospitalisé ? chambres libres ?

Angular via la Gateway
admissions, séjours, sorties

Vers MS7 : AdmissionCreee, SortieEnregistree
prévision d'occupation des chambres

En cas de panne

MS3 indisponible
sortie enregistrée, RDV « à planifier »

MS7 indisponible
statistiques mises à jour ensuite

Admission enregistrée
normalement dans MS6

Aucun blocage pour le patient

Pourquoi **Synchrone pour le rendez-vous de suivi, asynchrone pour les statistiques et prédictions.**

MS7 — Prédiction et recommandation (commun)

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Rôle



Prévoir ce qui va se passer dans les 7 prochains jours et proposer une solution.

Fonctions clés

- Prévoir la demande de médicaments
- Prévoir l'occupation des chambres
- Recommander une pharmacie proche
- Recommander un réapprovisionnement

Données clés

- Événement reçu
- Prévision
- Recommandation
- Modèle entraîné

Stockées dans la base MongoDB de MS7

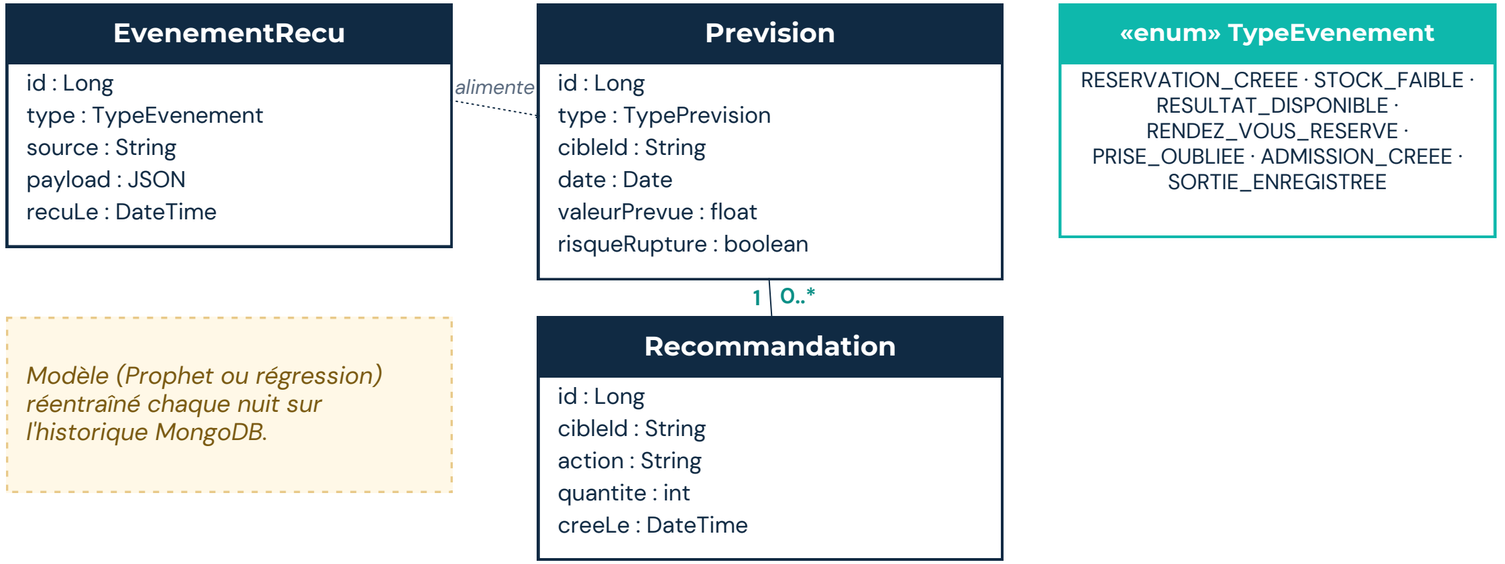
Objectif

Anticiper les ruptures de stock et le manque de chambres, et proposer une solution.

MS7 — Conception : classes et API

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Diagramme de classes



Endpoints et codes

- GET **/api/forecast/medications** — 200 · 404
- GET **/api/forecast/rooms?jours=7** — 200 · 400
- GET **/api/recommendations/pharmacies** — 200 · 404
- GET **/api/recommendations/restock** — 200 · 404

ÉVÉNEMENTS

Consomme : événements de MS1, MS2, MS3, MS4, MS6

Appelé par MS1 (prévision de demande)

Choix BD

MongoDB : historique d'événements JSON variés qui sert à entraîner le modèle de prédiction.

MS7 — Interactions

Python · FastAPI | MongoDB dédiée | Technologie avancée | Événements RabbitMQ

Appelle

Aucun appel sortant

MS7 lit ses événements et répond aux demandes de prévision

Appelé par · événements

Appelé par MS1 (synchrone)
prévision de demande d'un médicament

De MS1, MS2, MS3, MS4
ReservationCreee, ResultatDisponible, PriseOubliee…

De MS6
AdmissionCreee, SortieEnregistree

Angular (Admin) via la Gateway
prévisions et recommandations

En cas de panne

MS7 indisponible
messages gardés (files durables)

Au redémarrage
traitement, aucun événement perdu

MS1 affiche la fiche
sans prévision

Aucun blocage pour le patient

Pourquoi **Événements pour apprendre (asynchrone), un seul appel synchrone : la prévision demandée par MS1.**



Merci pour votre attention !

MediConnect